Improving Deployment Consistency of Cloud-Based Services: Manual Provisioning versus Terraform Automation

Meenakshi
24235415
Research in Computing - CA-1
National College of Ireland

1 Research Problem Background

Cloud computing has turned out to be the foundation of the contemporary online services where organizations are able to implement scalable application in a distributed environment. Ensuring that the deployment in the cloud service is consistent, i.e. the ability to be able to recreate the same infrastructure configuration in multiple environments, i.e. development, testing, and production is one of the biggest challenges in cloud service management. Typically, cloud infrastructure has been provisioned manually through provider consoles like AWS Management Console or Azure Portal. While manual provisioning may seem straightforward for small deployments, it often leads to configuration drift in state file, human errors, and inconsistent infrastructure setups. These inconsistencies can cause service outages, security vulnerabilities, and increased operational costs (Kasarla 2025)Akinbolaji et al. (2024).

Over the past few years, the solution to such problems is provided by so-called Infrastructure as Code (IaC) tools, like Terraform. Terraform is a technology that lets infrastructure be described in declarative configuration files, which can be automated, versioned, repeatable, and recoverable in case of failures much faster than traditional methods allow Pohjola (2025) Hayas (2022). Nevertheless, even though Terraform is gaining acceptance, most organizations continue to use manual provisioning because it lacks expertise or is seen as too complex. This is why it is necessary to research the question whether Terraform automation can substantially enhance deployment consistency in comparison with manual provisioning methods.

2 Research Question

To what extent can using Terraform to automate cloud infrastructure improve deployment consistency, reliability, and error reduction compared to provisioning cloud resources manually through the provider's console in a real cloud environment?

- How does manual provisioning of cloud infrastructure compare to Terraform-based automation in terms of configuration consistency across development, testing, and production environments?
- In what ways does Terraform reduce human errors and configuration drift when deploying cloud services, compared to traditional click-based/manual setups?
- How do deployment time, number of failed deployments, and rollback needs differ between manually provisioned infrastructure and Terraform-automated infrastructure?
- How does using Terraform for infrastructure as code support more reliable and repeatable deployments within DevOps/CI-CD workflows compared to manual approaches?
- Overall, how does the efficiency and stability of Terraform-based deployments compare with manually configured deployments for the same cloud-based service?

3 Justification

The suggested study has importance since the problem of deployment inconsistency has remained an urgent problem in cloud operations that impedes the effectiveness and dependability of contemporary IT systems. With manual provisioning of cloud resources, it is common in most organizations, particularly those that practice DevOps, to have unintentional configuration differences between environments like development, testing, and production. Such inconsistencies may cause unacceptable behavior of application, failure of deployment, and even security vulnerability. In addition, manual provisioning is time intensive and highly reliable on individual knowledge and experience such that it is hard to replicate the environment or even keep infrastructure consistent across time. These anthropocentric processes are less and less sustainable as the number and complexity of cloud systems increases. The solution to these issues is how Terraform, an Infrastructure as Code (IaC) tool, has become a promising tool of automating the provisioning and management of the infrastructure by defining it via codes. Using code of , Terraform makes deployments repeatable, versioned, and nearly devoid of configuration drift. This automation reduces the human error as well as enhancing consistency and efficiency at every phase of deployment. Moreover, the introduction of Terraform to the DevOps processes improves the lack of downtimes with continuous delivery and quicker recovery after the possible failures and increases the resilience of the service functioning. It is thus of great value to investigate the degree to which Terraform enhances the consistency of deployment as opposed to the manual approach. The results would inform organizations to make informed decisions based on data on IaC adoption and eventually enhance the stability, scalability, and operability of their cloud services. Do you want me to sound more official and academia or is the existing sound fine (understandable and business-like yet not too formal)?

4 Specific Items to Be Addressed

This study will focus on the following key investigation areas:

1. **Manual Provisioning Challenges**

   Investigating common issues such as inconsistency, state file drift, and error rates when infrastructure is created manually.

2. **Terraform Automation Benefits**

   Evaluating how Terraform improves repeatability, scalability, and infrastructure standards.

3. **Experimental Deployment Comparison**

   Deploying identical cloud-based services using both methods and comparing outcomes based on performance metrics.

4. **Consistency Measurement Metrics**

   Defining measurable indicators such as configuration reproducibility, time efficiency, and failure frequency.

5. **Impact on DevOps Practices**

   Assessing Terraform's role in CI/CD pipelines and modern DevOps workflows.

6. **Recommendations for Cloud Organizations**

   Providing best-practice guidance for organizations transitioning from manual provisioning to IaC automation.

References

Akinbolaji, T. J. et al. (2024), 'Automation in cloud-based devops: A guide to ci/cd pipelines and infrastructure as code (iac) with terraform and jenkins', *World Journal of Advanced Engineering Technology and Sciences* .
URL: *https://wjaets.com/sites/default/files/WJAETS-2024-0542.pdf*

Hayas, S. B. (2022), 'Importing terraform resources the safe way', `https://www.youtube.com/watch?v=VNBi-HhVX_Q`. YouTube video, accessed February 2026.

Kasarla, N. K. (2025), 'Implementing infrastructure as code (iac) with terraform for scalable cloud deployments', *Journal of Information Systems Engineering and Management* .

Pohjola, O. (2025), Mitigating configuration drift in infrastructure-as-code systems, Master's thesis, Aalto University.